

VDA11.3

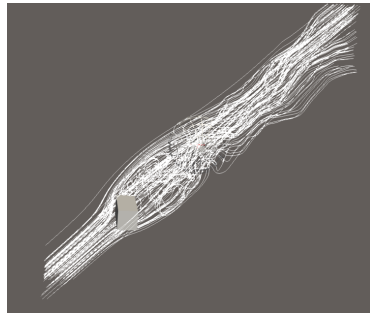
Hugo Lladró Prats

July 2026

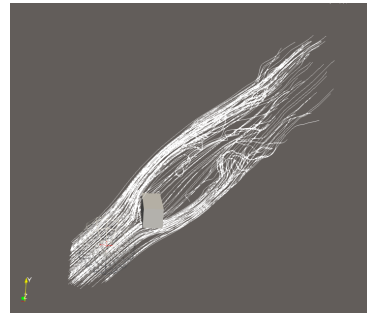
Exercise 3 (Vector Field Visualization in ParaView, 9 Points)

In this assignment, you will learn how to generate a geometry-based vector field visualization in ParaView.

a) Please open the files `cuboid velocity.vtk`, which contains a simulated flow around a cube, and `cube.vtp`, which represents the cube itself. Add a StreamTracer and set its seed type to “point source”. Compare results when seeding in front of the obstacle or behind it. Submit corresponding screen shots, one of which might look similar to Figure 1 (left). Do you see a reason to prefer one of the two seeding strategies over the other? (3P)



(a) Stream tracer with a point seed after the cube



(b) Stream tracer with a point seed before the cube

While the second figure has a more uniform representation with less clutter overall, if we are trying to understand the stream flow around the cube, the main regions of interest seem to be just behind the cube, and the first figure does a better job at highlighting these regions of high vorticity.

b) Turn the unshaded line rendering into shaded streamtubes that provide a better impression of their 3D trajectories, e.g., via a Tube filter. Color the streamlines based on velocity magnitude, similar to Figure 1 (right). Submit a screenshot. Can you identify regions in which the flow speeds up or slows down? (3P)

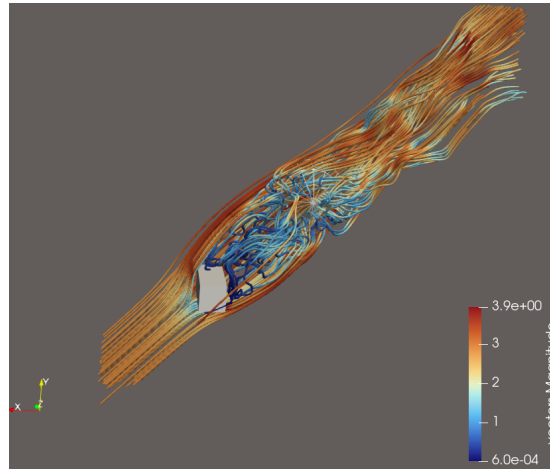


Figure 2: Stream tracer with the stream tubes setting and colored according to the magnitude of the velocity at each point

It seems that in the region directly behind the cube the air flow speeds and curls considerably, while the air flow that is displaced to the side by the cube speeds up. When it gets further from the cube the speed of the flow seems to be more uniform, even though still with some curl and regions of higher and lower speed.

c) There are different reasons due to which VTK might terminate streamline integration. Find out which of them applies to most of your streamlines by color coding them according to ReasonForTermination and looking up the interpretation of the respective numerical codes in the VTK documentation. How does your answer change when you substantially increase the maximum streamline length parameter? (3P)

We see that most of the lines have termination code 1, which corresponds to the line exiting the domain. A couple lines have code 3, which corresponds to an unexpected value. Some other lines have termination code 4, which corresponds to the line exceeding the maximum stream length.

Thus, if we increase the stream length we would expect for the lines that had termination code 4 to continue until they exit the domain and thus to have termination code 1.

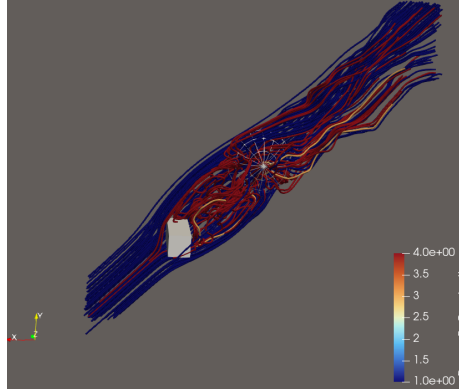


Figure 3: Stream tracer with coloring according to the termination code of each streamline

Indeed if we triple the maximum integration length we see that most of the lines that had termination code 4 now exit the domain in time. However we see also more lines with termination code 3, corresponding to receiving an unknown value.

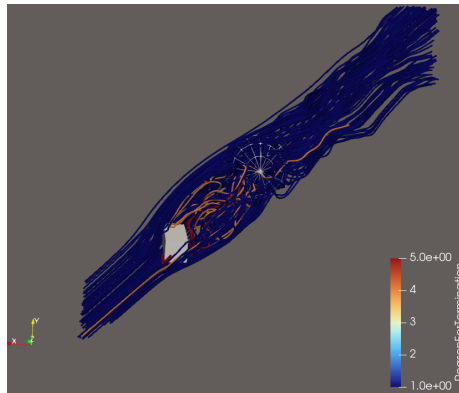


Figure 4: Steam traces showing termination codes with increased maximum stream length